Week 7:
Adapter Pattern

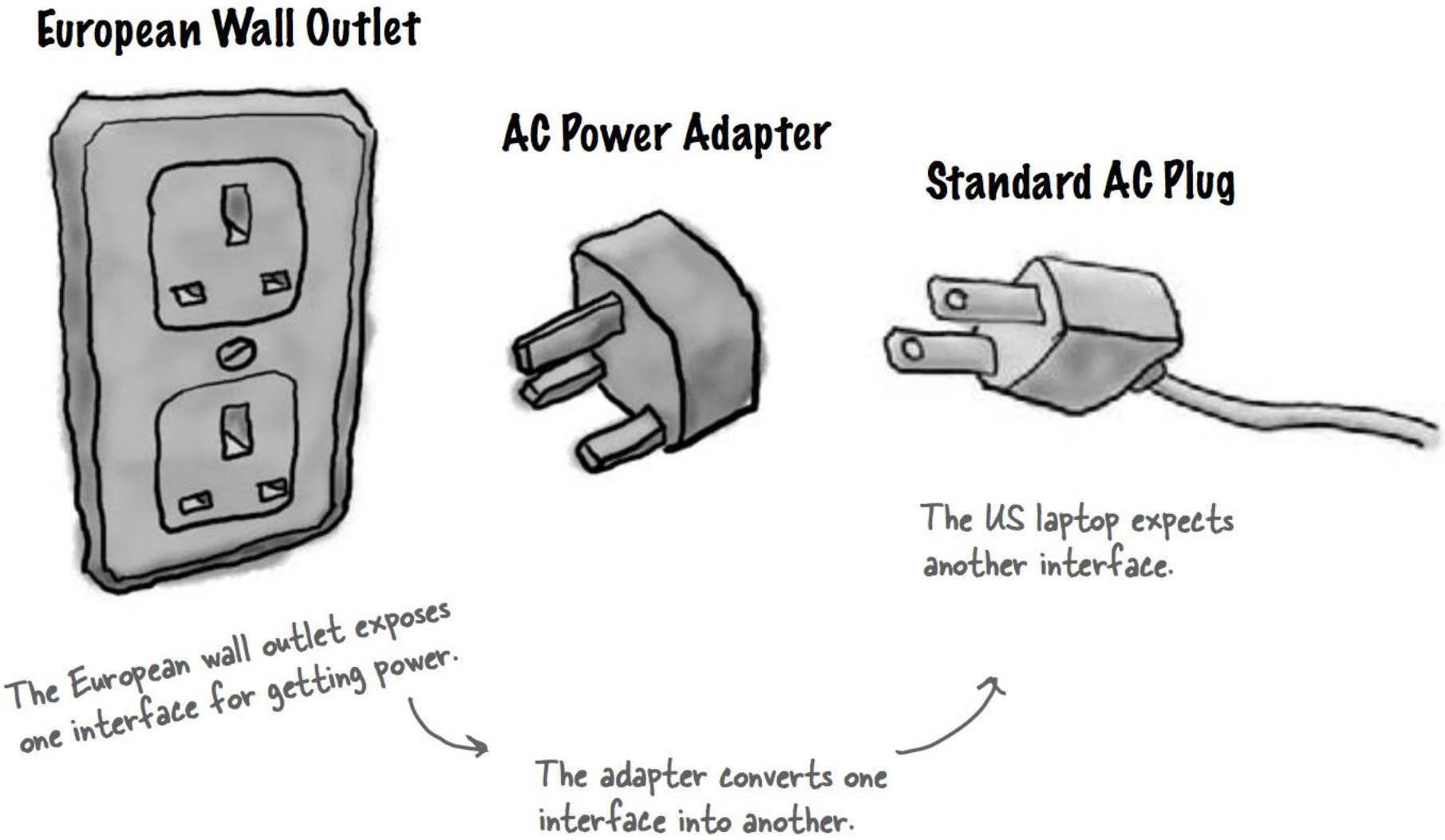
Agenda

- Adapter Pattern
  - An real life example
  - Adapter pattern definition
  - Software adapter example

Ví dụ

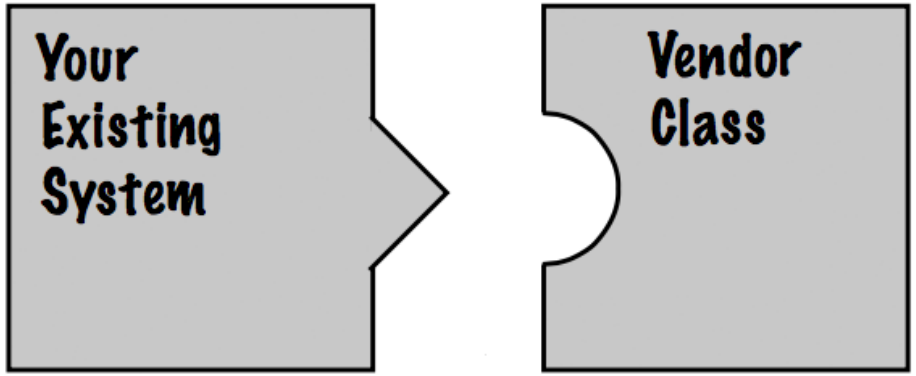
- Nêu các adapter gặp trong thực tế?

Adapters all around us



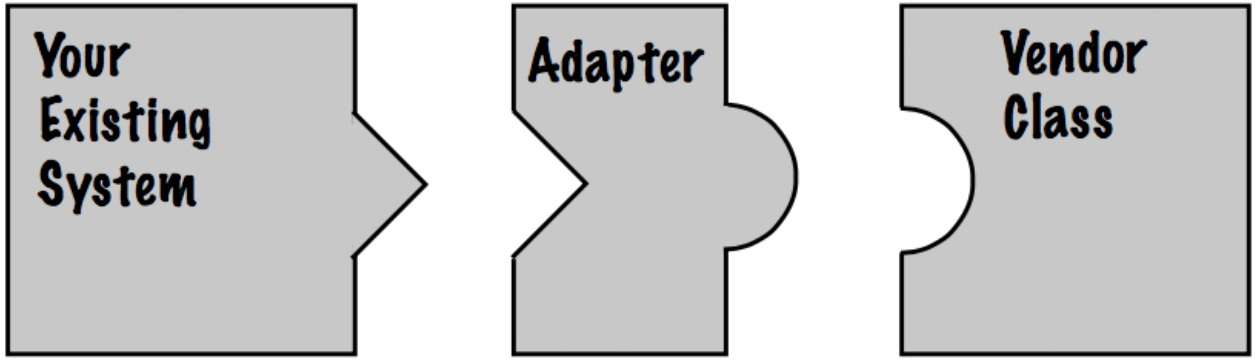
Object oriented adapters

- You've got an existing software system that you need to work a new vendor class library into, but the new vendor designed their interfaces differently than the last vendor:



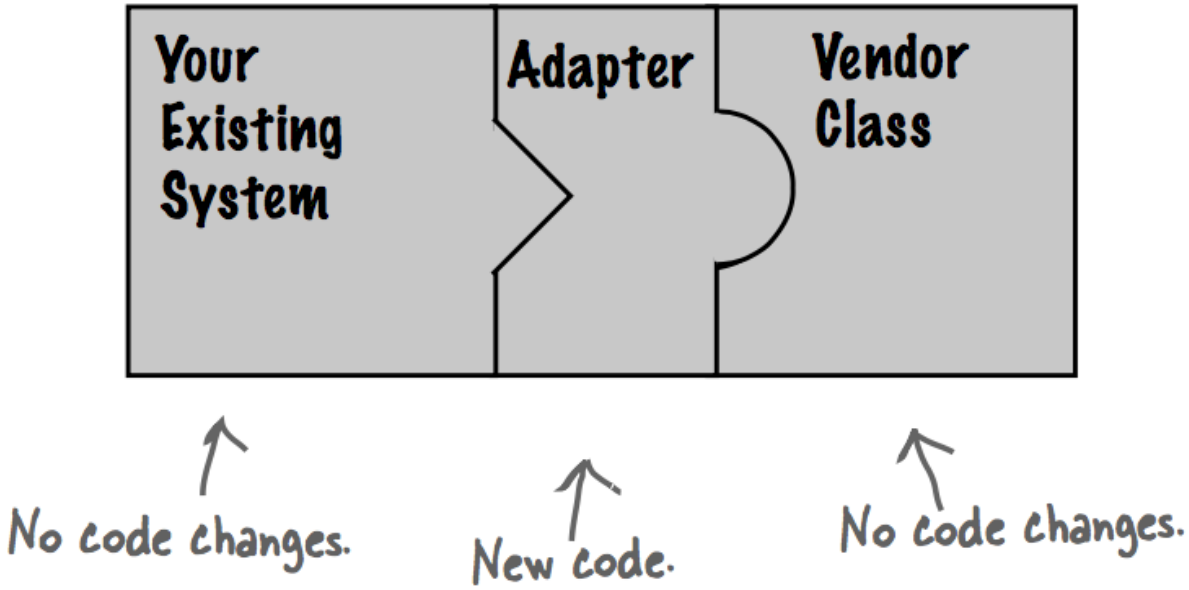
Object oriented adapters

- You don’t want to solve the problem by changing your existing code (and you can’t change the vendor’s code). So what do you do? Well, you can write a class that adapts the new vendor interface into the one you’re expecting.



Object oriented adapters

- The adapter acts as the middleman by receiving requests from the client and converting them into requests that make sense on the vendor classes.

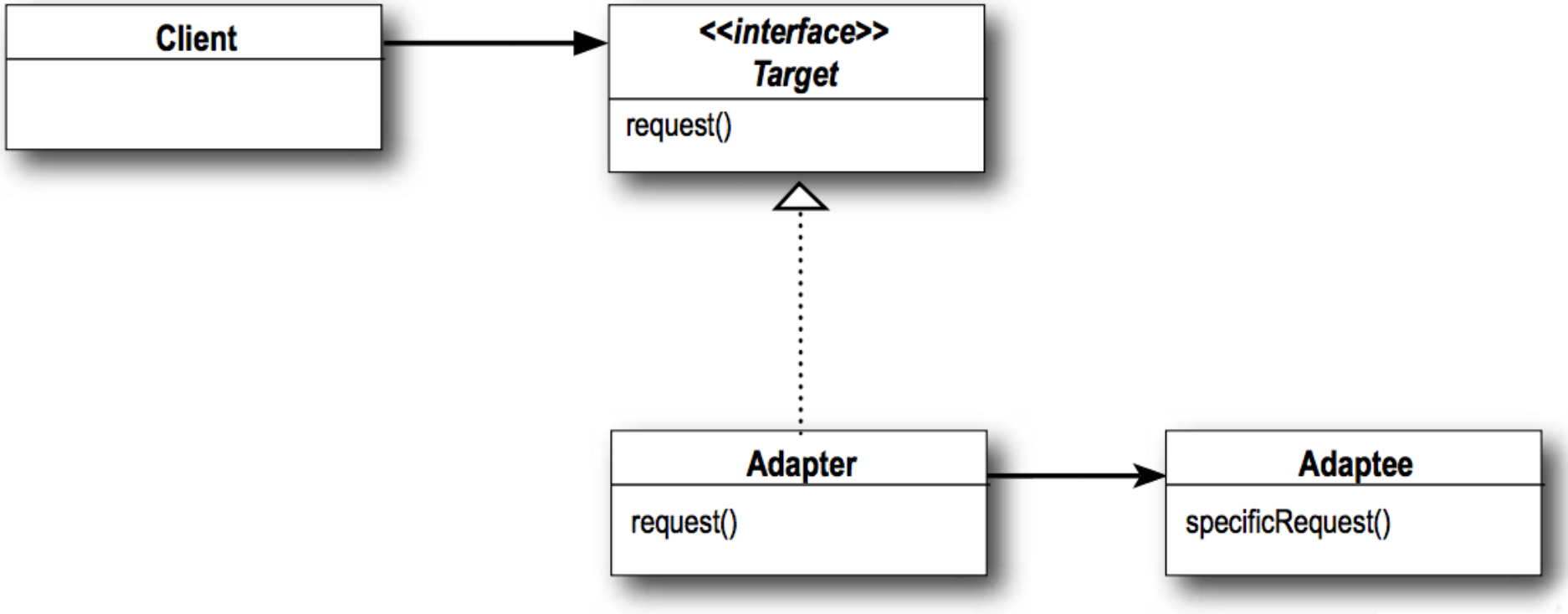


Adapter Pattern Definition

- GoF Intent: “Converts the interface of a class into another interface the clients expect. Adapter lets classes work together that couldn’t otherwise because of incompatible interfaces.”

- Also known as a “Wrapper”

Adapter Pattern Definition



Lab

We have an existing system that interacts with a simple FTP program. This component uses ISimpleFTP interface.

```
public interface ISimpleFTP
{
      //send a message
      void sendAMsg(string message);
      //make a connection to server
      void ConnectServer();
}
```

Lab

```
public interface ISimpleFTP
{
    //send a message
    void sendAMsg(string msg);
    //make a connection
    void connectServer();
}
class MyFTP implements
                    ISimpleFTP{
  public void sendAMsg(
                     string msg){
        //send a message
  }
  public void connectServer() {
        //make a connection
  }
}
```

```
class MyApp {
  ISimpleFTP ftp;
  public void setFTP( ...)
  public doSomething()
  {
      ftp.connectServer();
      ftp.sendAMsg("abc");
  }
}
class TestMyApp() {
  static void main(...){
     MyApp app = new MyApp();
     ISimpleFTP ftp =
                  new MyFTP();
     app.setFTP(ftp);
     app.doSomething();
  }
}
```

Lab

We just bought a new FTP DLL that comes from a different vendor and has some new functionality:

```
public interface IComplexFTP
{
    void sendMessages(string[] messages);
    void connect();
    string[] getDirs();
}
```

Lab

```
public interface IComplexFTP
{
  void sendMsgs(string[] msgs);
  void connect ();
  string[] getDirs();
}
class ComplexFTP implements
                IComplexFTP{
  public void sendMsgs(
                 string[] msgs
  {
      //send a message
  }
  public void connect() {
      //make a connection
  }
  public string[] getDirs() {
     //get directories
  }
}
```

```
class MyApp {
  ISimpleFTP ftp;
  public void setFTP( ...)
  public doSomething()
  {
      ftp.connectServer();
      ftp.sendAMsg("abc");
  }
}
class TestMyApp() {
  static void main(...){
     MyApp app = new MyApp();
     IComplexFTP ftp =
              new ComplexFTP();
     app.setFTP(ftp);
     app.doSomething();
  }
}
```

Lab

- We would like to use an adapter for the new DLL (IComplexFTP), but we need to support the old interface as well.
- Create an adapter that meet this requirement!
- Any solutions?

Answer: Two Way Adapter

```
public class FTPAdapter : ISimpleFTP
{
     private IComplexFTP complexFTP;

     public FTPAdapter()
     {
           complexFTP = new ComplexFTP();
     }
     public void sendAMsg(string msg){
         string[] msgs = new string[] { msg };
         complexFTP.sendMsgs(msgs);
     }
     public void connecServer(){
           complexFTP.connect();
     }
}
```

Lab

```
public interface IComplexFTP
{
  void sendMsgs(string[] msgs);
  void connect ();
  string[] getDirs();
}
class ComplexFTP implements
                 IComplexFTP{
  public void sendMsgs(
                  string[] msgs
  {
      //send a message
  }
  public void connect() {
      //make a connection
  }
  public string[] getDirs() {
     //get directories
  }
}
```

```
class MyApp {
  ISimpleFTP ftp;
  public void setFTP( ...)
  public doSomething()
  {
      ftp.connectServer();
      ftp.sendAMsg("abc");
  }
}
class TestMyApp() {
  static void main(...){
     MyApp app = new MyApp();
     ISimpleFTP ftp =
              new AdapterFTP();
     app.setFTP(ftp);
     app.doSomething();
  }
}
```